

The Chaos Crew

Assignment #9, Adversarial QA Agent. Rex Machina capstone.

An agent that runs against the shipped capstone build and tries to break it. It does not play the game. Run it with `node qa/adversary.js`. No key, no network, about twenty seconds.

What it does

The strategy is not random flailing. GDD §3 names six exploits that the v3 stress test found and closed, so the agent re-attempts them against the live build first, then falls back to invariants and a fuzzer for whatever nobody thought of.

Strategy	What it tries
Approach ratchet	Steps in and out to see whether the meter banks free progress. GDD §3 exploit 4.
The leash	Baits the robot with a movement pattern and checks whether it can be walked away from the dog. GDD §3 exploit 2 and §6.1.
Window aliasing	Runs a five-beat cycle, which never repeats inside the four-move window, and checks the engine scalars still catch it. GDD §3 exploit 5.
No fail state	Stands still for a whole fight and demands the game end it. GDD §3 exploit 3.
Read truth	Moves hard in one direction, then checks whether the line the robot speaks names the direction the dog actually favoured.
Invariant fuzz	300 randomised fights. Charge inside 0 to 100, both pawns on the grid, every spoken line present in a generated table, the fight always terminates.
Boundary probing	Holds into walls from the corner tile and checks the game reacts at all.
Encounter fairness	For all six Act 1 and 2 hazards, proves the documented escape works and that breaking early is punished.
Journey handoff	Searches a winning line, plays it after a clean journey and a failed one, and fails if Act 3 state is identical.
Difficulty parity	Re-attempts the closed exploits at all three settings, not just the middle one. The robot's memory is 3, 4 or 6 moves wide depending on it.
Solid pieces	Checks the two pawns never share a tile, and that walking into the robot is refused the way walking into a fence is: no round, no stamina, but something said.
Spawn fairness	Starts hundreds of random boards and asks whether a winning line exists at the stamina the player was given.
Planner honesty	Plays the build's own Watch the computer line and checks every move it commits to is legal, and that it wins.
Ledger truth	Turns on the move-grading judge and checks the grades against the position: no danger while a line survives, no excellent onto the tile the robot guessed.

Strategy	What it tries
Independent oracle	Re-searches positions the game has written off, using its own breadth-first search and a budget five times larger, and compares verdicts.

A run is about 3,000 checks. The count moves a little between runs because the board placer uses real randomness. `RM_SPAWNS`, `RM_PLANS` and `RM_ORACLE` in the environment change the sample sizes.

What it found

Three findings, one per severity. The full machine-readable output is `report_before_fixes.json`, in the required shape: `location`, `error_type`, `game_context`, plus severity, a repro, and the GDD clause each one violates.

RM-001, high, content logic mismatch. The robot told a dog that had moved right nine times out of nine: *"Target favored left nine of twenty recorded moves."* All three `sealing_direction` lines in `DT_NemesisReads.json` name "left", but the trigger fires whenever any one compass direction dominates. So the read was correct about the mechanic and wrong about the fact.

RM-002, medium, state underflow. Stamina reached -1 in 142 of 300 randomised fights. The round subtracts one for the round and one more for ending adjacent to the robot, and nothing floored it, so the HUD could show a negative number for a frame before the loss resolved.

RM-003, low, soft lock risk. Pressing into a wall did nothing at all. No round, no stamina, no robot move, nothing on screen. A player holding a key at the edge saw a frozen game and could not tell whether the input had registered.

All three are fixed in the shipped build. `report.json` is the same agent run against the current code: zero findings.

The build moved, and one check quietly died

After the agent was written the game gained difficulty settings, random boards, solid pieces, a search-driven demo mode and a panel that grades every move. That is most of the game's surface area, and none of it existed when the strategy list was drawn up. Re-running the agent proved nothing: it passed, because it was still attacking a game that had been replaced underneath it.

Worse, one check had rotted into a false pass. **Journey handoff** played a scripted line, seven right then four down, and compared the result after a clean journey and a failed one. That line only ever reached the kid because the dog could walk *through* the robot. Once the pieces became solid, two of its eleven moves were refused, both runs lost, and the check reported itself as passed while measuring nothing at all. A scripted expectation is a fossil. It now searches for a line instead of asserting one.

The six new strategies in the table above cover what the rewrite added. Three of them needed the game to expose seams it did not have: difficulty could not be set from outside, the search behind the demo mode was private, and the grading judge only ran when a browser document existed, which put it beyond the reach of any automated test. That last one is worth saying plainly: **a feature no test can reach is a feature nobody is checking**. The judge is a flag now.

What the extension proved

No new defects. Two results worth more than a defect would have been, both carried in `report.json` under `measurements`.

The fairness roll is load-bearing. A random board is placed, then rerolled until the game's own search can prove a win exists. That guarantee sounds like paperwork until you measure what it is holding back. Sampling raw placements with the roll disabled, against the same placement with it on:

Difficulty	Failed encounters	Raw board is winnable	After the fairness roll
easy	0	39 / 40	40 / 40
easy	3	39 / 40	40 / 40
moderate	0	35 / 40	40 / 40
moderate	3	26 / 40	40 / 40
hard	0	34 / 40	40 / 40
hard	3	25 / 40	40 / 40

On hard, after a bad journey, **three of every eight random boards would have handed the player a fight that could not be won**, with nothing on screen to say so. The roll is the only reason that never happens.

The harshest grade is honest. The ledger's worst verdict, *danger*, means the last winning line is gone. It is produced by a depth-bounded search, and a bounded search has two ways to come back empty: the position is lost, or it gave up. To the player those are the same sentence. So the agent does not ask the game. It replays a move from the exported primitives, runs its own breadth-first search with a far larger budget, and compares. Across the positions where the game called a run dead while the dog still had four or more rounds *and* four or more stamina in hand, the two searches agreed every time. Those runs really were lost.

Was I surprised

By RM-001, yes, and not because it was subtle.

That line was generated by the Assignment 4 RAG pipeline, which grounded it in the retrieved GDD chunk. It then passed the Assignment 6 evaluator, which checks five deterministic rules including one that rejects any number absent from the round's telemetry. It passed the Assignment 7 style evaluator, which scores tone, lore and format against the house style. Three separate pieces of machinery looked at that line and approved it, and all three were right to. The line is in voice, inside the word budget, cites a real observable, and invents no facts.

It is only wrong when the game is running. "Favored left" is a claim about state, and no check that reads the line in isolation can know whether the state agrees. That is the gap: every consistency check I built for this course validates content against a document. This one validated content against a running game, and it took eleven seconds to find something six weeks of static checking could not.

The fix follows the same idea. A line that names a compass direction is now held back unless that direction is the one the dog actually favoured, and the trigger ladder falls through to the next live read instead. So the robot says less and lies never.

The other two I would have found eventually by playing. This one I would have shipped, because when I played it by hand I happened to run left.

The second surprise came later, and it was about the agent rather than the game. A green run is not evidence. The agent went on reporting a clean pass through a rewrite that changed how movement, difficulty and scoring worked, and one of its checks was asserting something that had become impossible. The question to ask a passing suite is not whether it passed. It is when each check last had the chance to fail.

Files

qa/adversary.js	the agent
qa/report.json	the current run, 0 findings, plus measurements
qa/report_before_fixes.json	the run that found the three, kept as evidence